

Lab instructions:

Performance Profiling and Tuning using HPE Cray Perftools

Somrath Kanoksirirath

Scientific Support and Domain Research (SSD)

NSTDA Supercomputer Center (ThaiSC)

LAB-1 :: HPE Cray perftools

There is an example provided in `/project/thaisc/SSD/perftools/matmul/` on LANTA.

--> copy the entire directory to a local path

Inside `.../matmul/basic`, there are three job scripts—one for each variant.

Then,

1. Run the three job scripts yourself --> Are they for sampling or tracing experiments?
2. Edit the job scripts to run the other type of experiment
3. Try using the previously generated .apa file to run a tracing experiment
4. Building matmul.exe for a tracing experiments with `-DUSE_PAT_API=1` or `2` added --> What does it do?
5. Discuss the advantages and disadvantages of the sampling and tracing experiments
6. Discuss the advantages and disadvantages of the three perftools approaches.
7. Why does the python script (matmul.py) run much faster than the C++ program (matmul.exe)?

LAB-2 :: Matrix multiplication (matmul)

As in LAB-1, use the job scripts provided in your local `.../matmul/basic/`

Then,

1. Insert the below line to generate PAPI reports (one at a time)

Instruction

```
export PAT_RT_PERFCTR="PAPI_TOT_CYC,PAPI_TOT_INS,PAPI_VEC_INS"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_FP_INS,PAPI_FP_OPS"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_BR_INS,PAPI_BR_MSP"
```

Cache

```
export PAT_RT_PERFCTR="PAPI_L1_DCA,PAPI_L1_DCM,PAPI_L2_DCM"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_TOT_INS,PAPI_TLB_DM"
```

then calculate the metrics mentioned in this section

2. Identify the major bottlenecks of the unoptimized matmul.exe program, by comparing with matmul.py
3. Adding -O3 flag when building matmul.exe, then repeat 1 and 2 again

LAB-3 :: Single-core optimization

Use the matrix multiplication code in your local `.../matmul/single/`

Inside there are two job scripts—one is a template; another is for a reference BLAS dgemm run.

Problem:

1. Number of repeats (or iterations) ≥ 100 (but you may start with just 10)
2. Matrix size: $M = 3,000 \pm 100$, $N = 3,000 \pm 100$, $P = 3,000 \pm 100$
 while $M*N*P \geq 27,000,000,000$ (but you may start with a smaller one)
3. Use only one single CPU core (that is, no OpenMP, no MPI, no multi-core parallelism, ...)
4. Use double precision FP datatype (64-bit, double)
5. Without file IO, that is, `-DWRITE_ARRAYS=0`

LAB-3.1 :: Single-core optimization -- cache

Then,

1. Optimize USE_ALGOR=2 and 3 respectively by adjusting the compile-time parameters in .../matmul/src/matmul_setup.hpp while using -O3

```
// *** Block size (for ALGOR=2,3) ***
constexpr int  BLOCK_M_SIZE = 1000 ;
constexpr int  BLOCK_N_SIZE = 1000 ;
constexpr int  BLOCK_P_SIZE = 1000 ;

// *** Micro size (for ALGOR=3) ***
constexpr int  MICRO_M_SIZE = 2 ;
constexpr int  MICRO_P_SIZE = 2 ;
```

Then, calculate the PAPI metrics of your final programs.

Note: you may use any compiler but your final program using USE_ALGOR=3 should run roughly 5% slower than the corresponding Intel MKL dgemm run

LAB-3.2 :: Single-core optimization -- instr.

Using the same matrix problem in Lab 3.1,

1. Use **-DUSE_ALGOR=3** without the manually vectorized kernel **-DALGOR3_NO_MANUAL_VEC_KERNEL=1** and the parameters tuned in Lab 3.1, then optimize the program by using compiler flags and/or adding compiler directives to the `micro_kernel_scalar()` function in `.../matmul/src/matmul_kernel.hpp`
 2. (Optional) Repeat Lab 3.1 and 3.2 but for single precision FP datatype (32-bit, float) instead
 --> Then, compare and discuss the differences in performance, PAPI statistics and limitations
-
3. Enable file IO, i.e., **-DWRITE_ARRAYS=1**, and use **pat_build -g syscall,sysio,sysfs,lustre,io,dl -u ...** then
 - a) Use the `.../matmul/src/check.py` script to cross-check the output
 - b) Repeat again using **-DUSE_TXT_FORMAT=1** then compare writing in binary and in text format, both runtime and file size
 - c) Vary **-DNUM_ITER_PER_WRITE=...** and note on the IO time

LAB-4.1 :: Parallel programming -- OpenMP

Use the matrix multiplication code in your local `.../matmul/parallel/`
 Inside there are three example job scripts for OpenMP, MPI and GPU.

For the following tasks, use ≥ 100 iterations, $M = 10,000 \pm 500$, $N = 10,000 \pm 500$, $P = 10,000 \pm 500$ and employ both your optimized (`-DUSE_ALGOR=3`) and a BLAS implementation (`-DUSE_ALGOR=4`)

[OMP] => For each task, changing between `-DUSE_OMP=1` and `-DUSE_OMP=2`

1. What is the additional information in the PAT report generated when using `pat_build -g omp,threads,blas,dl -u` with either Cray CCE or Intel oneAPI compilers
2. Which configuration pair of `-DUSE_ALGOR` and `-DUSE_OMP` is the fastest? Explain why?
3. Repeat Step 2 with `-DUSE_TIMER=0 -DPRINT_LOOP_ITER=0`, then discuss
4. Repeat Step 3 but changing `OMP_SCHEDULE` to some other values
5. Vary the number of threads --> Discuss on their scalability

LAB-4.2 :: Parallel programming -- MPI

[MPI] => Using the same problem as in Lab 4.1 and changing between *-DUSE_MPI=1* and *-DUSE_MPI=2*

1. What is the additional information in the PAT report appeared when using *pat_build -g mpi,lustre,io,sysio,dl -u* with *-DWRITE_ARRAYS=1*
2. Vary the number of MPI tasks, without file IO *-DWRITE_ARRAYS=0*
 - a) Which is faster? *-DUSE_MPI=1* or *-DUSE_MPI=2*? Explain why
 - b) Does it depend on the number of MPI tasks?
 - c) Find the local matrix sizes and the order of *MPI_Bcast* and *MPI_Allgather* message size? Any comments?
3. Vary the number of MPI tasks and *-DUSE_MPI_IO=0,1,2 -DWRITE_ARRAYS=1*
 - a) Which combination is the fastest? Compare the reports then explain why
 - b) Does it depend on the number of MPI tasks? Matrix size?
 - c) Randomly check the output files using the *.../matmul/src/check.py* script as well

LAB-4.3 :: Parallel programming -- Hybrid

[Hybrid] => Using the same problem as in Lab 4.1

1. Write a hybrid (OpenMP+MPI) job script and come up with an appropriated set of pat_build options
2. Find the most optimal configuration of *-DUSE_OMP*, *-DUSE_MPI*, *number of threads* and *number of tasks*, for *-DUSE_ALGOR=3* in your opinion
3. Repeat Step 2 but for *-DUSE_ALGOR=4* --> Is it the same?
4. From your previous findings, discuss and guess why the configuration using *-qmkl=parallel* is much faster than the rest in Lab 4.1
5. Enable file IO, *-DUSE_MPI_IO=0,1,2* *-DWRITE_ARRAYS=1* --> check the output files using the script
6. Make suggestions on how to improve the matmul program --- What are its current limitations?

LAB-4.4 :: Parallel programming -- GPU

[GPU] => Using the `.../matmul/parallel/submitGPU.sh`

1. What is the additional information in the PAT report when using `pat_build -g cuda,cuda_math,blas,mpi,dl`
2. Gradually increase the problem size
 - a) At what problem size does GPU start spending more time computing than transferring/waiting for data?
 - b) Find one of the largest cases that can still run on **a single GPU card (40 GB VRAM)**
3. Running a case using **multiple GPU cards** (using `-DUSE_MPI=1,2` but keep the message size under 1 GB!)
 - a) Discuss on the bottleneck and the limitation found
 - b) Search for a solution or a better approach online

Lab 5.1 :: Apprentice

Install Apprentice 2 or 3 on your local machine. The installers are at \$CRAYPAT_ROOT/share/desktop_installers/.

Then, with **export PAT_RT_SUMMARY=0** set

1. Run hybrid matmul programs with file IO using

- -DUSE_ALGOR=3
- -DUSE_ALGOR=4 -DSELECT_BLAS=1 (MKL)
- -DUSE_ALGOR=4 -DSELECT_BLAS=0 (cray-libsci)

then, open the cases with Apprentice on your local machine and compare the charts

2. Run a GPU matmul program, then open the case with Apprentice and discuss the charts